In [1]:
```
# Name - Rohan Pise
# Rollno. - 01
```

In [3]:
```
import numpy as np

# Activation function (Sigmoid) and its derivative
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)  # Derivative of sigmoid

# Training data (XOR problem)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])  # Input features
y = np.array([[0], [1], [1], [0]])  # Target labels

# Initialize weights and biases randomly
np.random.seed(1)
input_neurons, hidden_neurons, output_neurons = 2, 4, 1

weights_input_hidden = np.random.rand(input_neurons, hidden_neurons)
weights_hidden_output = np.random.rand(hidden_neurons, output_neurons)
bias_hidden = np.random.rand(1, hidden_neurons)
bias_output = np.random.rand(1, output_neurons)

# Training process
epochs = 10000
learning_rate = 0.1

for epoch in range(epochs):
    # **Forward Propagation**
    hidden_input = np.dot(X, weights_input_hidden) + bias_hidden
    hidden_output = sigmoid(hidden_input)  # Activation at hidden layer

    final_input = np.dot(hidden_output, weights_hidden_output) + bias_output
    final_output = sigmoid(final_input)  # Activation at output layer

    # **Backpropagation**
    error = y - final_output  # Compute error
    d_output = error * sigmoid_derivative(final_output)  # Output layer gradient

    error_hidden = d_output.dot(weights_hidden_output.T)
    d_hidden = error_hidden * sigmoid_derivative(hidden_output)  # Hidden layer gra

    # **Update Weights and Biases**
    weights_hidden_output += hidden_output.T.dot(d_output) * learning_rate
    weights_input_hidden += X.T.dot(d_hidden) * learning_rate
    bias_output += np.sum(d_output, axis=0, keepdims=True) * learning_rate
    bias_hidden += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

    # Print loss every 1000 epochs
    if epoch % 1000 == 0:
        loss = np.mean(np.abs(error))
        print(f"Epoch {epoch}, Loss: {loss:.4f}")
```

```
# **Testing**
print("\nFinal Predictions:")
for i in range(len(X)):
    hidden_layer = sigmoid(np.dot(X[i], weights_input_hidden) + bias_hidden)
    output = sigmoid(np.dot(hidden_layer, weights_hidden_output) + bias_output)
    print(f"Input: {X[i]} => Predicted: {output[0][0]:.4f}")
```

```
Epoch 0, Loss: 0.4994
Epoch 1000, Loss: 0.4997
Epoch 2000, Loss: 0.4990
Epoch 3000, Loss: 0.4944
Epoch 4000, Loss: 0.4553
Epoch 5000, Loss: 0.3682
Epoch 6000, Loss: 0.2284
Epoch 7000, Loss: 0.1335
Epoch 8000, Loss: 0.0952
Epoch 9000, Loss: 0.0757

Final Predictions:
Input: [0 0] => Predicted: 0.0650
Input: [0 1] => Predicted: 0.9429
Input: [1 0] => Predicted: 0.9331
Input: [1 1] => Predicted: 0.0662
```

In [ ]: